Fundamentos de Sistemas Embebidos

Unidades 1 y 2 · Apuntes y resumen de estudio para el examen

PARTE 1 · Apuntes completos

Unidad 1: Introducción a los Sistemas Embebidos

1.1 Concepto de sistema y sistema embebido

En ingeniería, un **sistema** es un conjunto de elementos interconectados o interrelacionados que realizan una función o cumplen un propósito específico. Algunas precisiones importantes:

- No todo sistema tiene entradas ni salidas explícitas.
- No todo sistema es electrónico ni digital: existen sistemas biológicos, mecánicos, térmicos y financieros.
- Pueden ser **abiertos** (interactúan con su entorno) o **cerrados**.

Un **sistema embebido (integrado)** es un sistema de cómputo de propósito específico, con características de hardware y software más restringidas que las de una computadora personal o una supercomputadora.

Características clave:

- Diseñado para una función dedicada y predefinida.
- Exigencias de calidad, confiabilidad, estabilidad y robustez superiores a las de una PC de propósito general.
- Opera bajo restricciones físicas estrictas: consumo de energía, disipación de calor, espacio físico y costo unitario de fabricación (BOM cost).

1.2 Ejemplos de sistemas (físicos, mecánicos y electrónicos)

- **Generales:** sistema financiero, sistema de salud pública, cuerpo humano, bacteria, automóvil, controlador PID, computadora.
- **Físicos / mecánicos:** lavadoras electromecánicas tradicionales con temporizadores mecánicos de leva, o semáforos mecánicos antiguos. Demuestran que no se necesita procesamiento digital para tareas mecánicas complejas.
- **Electrónicos y digitales embebidos:** medidores de consumo eléctrico inteligentes, controladores de motor automotriz (ECU), marcapasos médicos, módulos IoT y sistemas de control industrial.

1.3 Arquitectura de los sistemas embebidos

La **abstracción arquitectónica** es un modelo formal por capas que permite estructurar la interacción entre hardware y software sin manejar toda la complejidad física al mismo tiempo.

Componentes de hardware (HW):

- **Procesador / núcleo:** Unidad de Control (UC), Unidad Aritmético-Lógica (ALU) y registros.
- **Memoria:** volátil para datos temporales y no volátil para el código del programa.

- **Periféricos y E/S:** sensores, actuadores, temporizadores (timers), controladores de interrupciones (ICU) e interfaces de comunicación.

Componentes de software (SW):

- **Firmware / bare-metal:** código ejecutado directamente sobre el hardware, sin sistema operativo.
- **RTOS / kernel de SO:** gestiona el calendarizador de tareas (scheduler), la asignación de memoria y la abstracción de periféricos.

1.4 Importancia y tipos de diseño

Desafíos principales:

- Restricciones severas de costo unitario (BOM cost).
- Tolerancia a fallas y cumplimiento de estándares de seguridad.
- Limitaciones físicas de energía, calor y tamaño.
- Reducción del tiempo de salida al mercado (Time-to-Market).

Modelo de diseño	Descripción	Riesgo / característica
Big-Bang	Desarrollo sin planificación previa; se integran los componentes esperando que funcionen.	Alto riesgo de fallo
Programa y corrige (Code & Fix)	Desarrollo empírico de ensayo y error con requerimientos mal definidos.	Poco control y calidad
Basado en arquitectura	Planificación rigurosa con modelos de abstracción; resuelve las restricciones físicas y funcionales desde el diseño.	Modelo recomendado

1.5 Modelos, tipos y tamaños

Clasificación por tecnología:

- **Sistema operativo:** sin SO (bare-metal), RTOS (Real-Time Operating System) o Linux embebido.
- **Lenguaje:** C, C++, ensamblador o lenguajes interpretados/asistidos (Python/MicroPython).
- **Hardware:** arquitecturas de 8, 16, 32 o 64 bits, o SoCs híbridos con lógica programable (FPGA).

Tamaño	Base de hardware	Características
Pequeño / micro	Microcontroladores de 8/16 bits (PIC18, ATMega328p)	RAM en bytes/kilobytes; procesamiento directo
Mediano	Microcontroladores de 32 bits (ESP32, RP2040, ARM Cortex-M)	Flash/RAM en megabytes; conectividad serie e inalámbrica
Grande / complejo	Procesadores de aplicación de 32/64 bits (ARM Cortex-A, p. ej. Raspberry Pi)	Ejecutan SO completos (Linux, Android) con memoria virtual (MMU) y paginación

Unidad 2: Organización, Arquitectura y Plataformas de Sistemas Embebidos

2.1 Procesador y evolución de arquitecturas

Un **procesador/computadora** es una unidad central compuesta por la Unidad de Control (UC), la Unidad Aritmético-Lógica (ALU), registros de almacenamiento, espacio de memoria e interfaces de E/S.

Taxonomía de Flynn (según el flujo de instrucciones y de datos):

Clase	Significado	Ejemplo / aplicación
SISD	Single Instruction, Single Data	Procesador secuencial de un solo núcleo
SIMD	Single Instruction, Multiple Data	Una instrucción opera sobre varios datos a la vez: procesamiento vectorial / GPU
MISD	Multiple Instruction, Single Data	Sistemas redundantes críticos
MIMD	Multiple Instruction, Multiple Data	Ejecución paralela multinúcleo y sistemas distribuidos

Filosofías CISC vs. RISC:

Aspecto	CISC (Complex Instruction Set Computer)	RISC (Reduced Instruction Set Computer)
Instrucciones	Complejas y de tamaño variable	Conjunto reducido y de tamaño fijo (32 bits)
Direccionamiento	Múltiples modos	Pocos modos; arquitectura Load/Store (solo las instrucciones de carga y almacenamiento acceden a RAM)
Ciclos	Varios ciclos de reloj por instrucción	Optimizada para segmentación (pipeline)
Registros	Uso moderado	Alto uso de registros
Ejemplo	Intel x86	ARM

Evolución histórica: de los procesadores de 4 bits (Intel 4004) a los de 16 bits (8086) y 32 bits (x86 / IA-32). Después, el salto a 64 bits (x86-64 / AMD64) manteniendo retrocompatibilidad mediante los modos de operación real, protegido y largo.

2.2 Mapa de memoria del sistema

El **mapa de memoria** es la organización y asignación lógica de direcciones donde el procesador ubica la memoria de programa, la memoria de datos y los registros de periféricos/E/S.

Modelo	Descripción
von Neumann	Bus único para datos e instrucciones; ambos comparten el mismo espacio de memoria.
Harvard	Buses físicamente separados para datos e instrucciones; permite leer y ejecutar de forma simultánea.

Plataformas estudiadas:

Plataforma	Características principales
PIC18F452	Microcontrolador de 8 bits, arquitectura Harvard. Memoria de programa direccionable hasta 2 MB, con vectores de RESET (0000h), alta prioridad (0008h) y baja prioridad (0018h). Memoria de datos dividida en bancos (bank switching) mediante el registro BSR, más un banco de acceso directo (Access Bank).
ATMega328p (Arduino Uno)	Arquitectura AVR de 8 bits, Harvard. Mapa de datos: 32 registros de propósito general (00h–1Fh), 64 registros de E/S (20h–5Fh), E/S extendida y SRAM interna (desde 0100h).
ESP32 (Xtensa LX6 / RISC-V)	SoC de 32 bits con bus de instrucciones (I-bus) y bus de datos (D-bus) independientes. Mapea RTC FAST/SLOW SRAM, Flash interna y soporte de Flash/PSRAM externa mediante ejecución directa en sitio (XIP).
Raspberry Pi Pico (RP2040)	Doble núcleo ARM Cortex-M0+ (ARMv6-M) de 32 bits. 264 KB de SRAM interna en bancos distribuidos, Flash externa por bus QSPI con caché XIP y bloques de periféricos programables (PIO).

2.3 Controlador de interrupciones (ICU / APIC)

Una **interrupción** es un evento de hardware o software que interrumpe el flujo secuencial de ejecución para que el procesador atienda una señal externa u origen específico mediante una **Rutina de Servicio de Interrupción (ISR)**.

Tipos de interrupciones:

- **Enmascarables (ignorables):** se habilitan o deshabilitan con bits de máscara en registros de control, por ejemplo durante zonas críticas de código.
- **No enmascarables (NMI):** eventos críticos que el procesador debe atender siempre (falla de voltaje, sobrecalentamiento, Reset).

Niveles de prioridad:

- **Mononivel:** todas las interrupciones tienen la misma prioridad; si llega una nueva mientras se atiende otra, debe esperar a que termine la primera.
- **Multinivel:** prioridades jerárquicas; una interrupción de mayor prioridad puede interrumpir una ISR de menor prioridad en curso (anidamiento).

Controladores avanzados:

- **NVIC** (Nested Vectored Interrupt Controller, ARM Cortex-M): manejo vectorial con latencia ultrabaja y prioridad anidada por hardware.
- **APIC / LAPIC** (x86/Intel): controlador local programable para arquitecturas distribuidas y multiprocesador.

2.4 Temporizadores (timers de hardware)

> **Definición exigida en examen**
> Un temporizador o timer en hardware es un **contador en cuenta regresiva** que **activa una interrupción al llegar a cero** y **reinicia el conteo automáticamente**.

Importancia: es el "reloj de pared" o alma del procesador y del multitasking; es esencial para que el scheduler del SO conmute tareas y gestione retrasos exactos.

Modo	Funcionamiento
Free-Running	Conteo continuo; al llegar a cero reinicia automáticamente y sigue contando indefinidamente.
One-Shot	Se activa ante un evento específico, cuenta una sola vez hasta la meta, dispara la interrupción y se detiene automáticamente.

Características de un timer: dirección de conteo (incrementa/decrementa), rango (número de bits) y fuente de reloj/prescaler.

2.5 Memoria volátil, no volátil y modos de direccionamiento

Memoria volátil: conserva los datos solo mientras haya suministro eléctrico.

- **SRAM (Static RAM):** construida con flip-flops de transistores; muy rápida, no requiere refresco y tiene mayor consumo. Se usa en registros, memoria interna de microcontroladores y cachés.
- **DRAM (Dynamic RAM):** construida con capacitores que pierden su carga; requiere ciclos constantes de refresco (aprox. cada 60 ms) para que la información no se destruya.

Memoria no volátil: conserva la información de forma permanente sin energía eléctrica (ROM, EEPROM, Flash).

Modos de direccionamiento (modelo base Motorola 68HC11):

Modo	Descripción	Ejemplo
Inherente / implícito	No requiere operandos explícitos; el operando está definido en la propia instrucción.	NOP
Inmediato	El valor del dato está contenido en la propia instrucción.	MOV #0x05
Directo	Apunta a una dirección corta en la página cero de memoria.	—
Extendido	Contiene la dirección de memoria completa (lógica o física).	—
Indexado	La dirección se calcula con un registro índice más un desplazamiento.	—
Relativo	La dirección destino se calcula respecto a la posición actual del Contador de Programa (PC).	Saltos condicionales

2.6 Bus Serie Universal (USB)

Topología: estructura jerárquica en árbol/pirámide (estrella) controlada por un host central mediante concentradores (hubs), con un límite estricto de hasta 7 niveles de profundidad.

Velocidad (USB 2.0)	Tasa	Dispositivos típicos
Low Speed	1.5 Mbps	Interfaz humana (teclados, ratones)

Velocidad (USB 2.0)	Tasa	Dispositivos típicos
Full Speed	12 Mbps	Audio o datos estándar
High Speed	480 Mbps	Unidades de almacenamiento, cámaras

- **Capa física:** señales diferenciales en par trenzado (D+ y D−), con inmunidad frente al ruido electromagnético.
- **Alimentación:** al conectarse, la especificación asegura 100 mA a 5 V por defecto; consumos mayores se negocian por software.

Tipos de transferencia:

Tipo	Uso
Control	Configuración e inicialización del dispositivo.
Bulk	Grandes volúmenes de datos con detección de errores, pero sin latencia garantizada.
Interrupt	Transferencias pequeñas y periódicas con latencia garantizada.
Isochronous	Tiempo real a tasa constante, sin retransmisión por errores (audio/video).

2.7 Interconexión de dispositivos y buses de comunicación

I²C (Inter-Integrated Circuit)

- Bus síncrono de 2 líneas: SDA (datos) y SCL (reloj), a colector abierto con resistencias pull-up.
- Arquitectura maestro-esclavo con direcciones de 7 o 10 bits.
- **START:** SDA pasa de alto a bajo con SCL en alto. Transmisión byte a byte con bit de confirmación (ACK/NACK). **STOP:** SDA pasa de bajo a alto con SCL en alto.

UART (Universal Asynchronous Receiver-Transmitter)

- Protocolo asíncrono punto a punto con líneas dedicadas de transmisión (Tx) y recepción (Rx).
- **Trama N81** (la más usada): 1 bit de inicio, 8 bits de datos, sin paridad y 1 bit de parada.
- **Trama real:** aunque lleva 8 bits de datos, en el canal se envían **10 bits** (o 10.5 con 1.5 bits de stop) por los bits de encuadre.
- **Control de flujo por hardware:** líneas RTS (Request to Send) y CTS (Clear to Send). **Por software:** caracteres XON (CTRL+S) y XOFF (CTRL+Q).

Otros estándares comparados:

Estándar	Características
SPI	Bus síncrono de 4 hilos (MOSI, MISO, SCK, CS); ideal para alta velocidad a corta distancia.
RS-232 / RS-485	RS-485 usa líneas diferenciales: inmunidad al ruido en distancias largas y redes multipunto.
One-Wire	Un solo hilo de datos más tierra; permite alimentar dispositivos con un capacitor (parasitic power) y direccionar sensores con un ID único de 64 bits en ROM.

2.8 Secuencia de arranque (bootstrap)

El **vector de RESET** es la dirección de memoria fija donde el procesador hace el primer fetch de instrucción justo después de encender o reiniciar.

Tipos de RESET:

- **Hard RESET (Power-On Reset, POR):** restablecimiento físico inicial al aplicar energía eléctrica.
- **Soft RESET:** provocado por software, desbordamiento del Watchdog Timer, sobrecalentamiento (overheat) o eventos de suspensión de energía (ACPI).

Pasos de la secuencia de arranque:

1. **Estabilización de energía y reloj:** se regulan los voltajes de la fuente y las señales del oscilador.
2. **Fetch del vector de RESET:** el procesador obtiene la dirección del código de inicio (BIOS/UEFI/firmware).
3. **Designación de núcleos (Intel / multinúcleo):** el LAPIC de cada procesador se reinicia. El bit BSP del LAPIC se pone en 1 para el **Bootstrap Processor (BSP)** y en 0 para todos los **Application Processors (AP)**.
   - El BSP asume la función maestra: ejecuta el fetch, carga el BIOS/UEFI, inicializa el APIC y coordina la carga del SO.
   - Los AP ejecutan una autoprueba básica y esperan en estado de espera hasta recibir interrupciones de inicio (SIPI) enviadas por el BSP.
4. **Prueba, copia de memoria y carga del kernel:** se inicializa la DRAM, se realiza la autoprueba POST, se copian las rutinas críticas a RAM y se transfiere el control al kernel del SO.

PARTE 2 · Resumen para estudiar

Versión corta de los puntos que más probablemente se preguntan. Léela primero y consulta la Parte 1 si algo no te queda claro.

Unidad 1

Un **sistema** es un conjunto de elementos interrelacionados con un propósito; puede ser abierto o cerrado y no tiene por qué ser electrónico. Un **sistema embebido** es un sistema de cómputo de propósito específico, más restringido que una PC, que hace una función dedicada con mayor confiabilidad y robustez y bajo límites de energía, calor, tamaño y costo (BOM).

Su arquitectura se divide en **hardware** (procesador con UC, ALU y registros; memoria volátil y no volátil; periféricos, timers, ICU) y **software** (firmware bare-metal o RTOS/kernel con scheduler).

Los tres modelos de diseño van de peor a mejor así: **Big-Bang** (sin planificar), **Code & Fix** (ensayo y error) y **basado en arquitectura** (planificado con abstracción).

Tamaño	Ejemplos	Rasgo clave
Pequeño	PIC18, ATMega328p (8/16 bits)	RAM en bytes/KB
Mediano	ESP32, RP2040, Cortex-M (32 bits)	Flash/RAM en MB; conectividad
Grande	Cortex-A / Raspberry Pi	Linux/Android, MMU, paginación

Unidad 2

Arquitecturas. Flynn: SISD (un núcleo), SIMD (vectorial/GPU), MISD (redundancia), MIMD (multinúcleo). **CISC** tiene instrucciones complejas y variables (x86); **RISC** tiene instrucciones simples de 32 bits, Load/Store, muchos registros y pipeline (ARM). **von Neumann** usa un solo bus; **Harvard** separa los buses de datos e instrucciones.

Plataformas. PIC18F452 y ATMega328p: 8 bits, Harvard. ESP32: 32 bits, I-bus y D-bus separados, XIP. RP2040: doble Cortex-M0+, 264 KB de SRAM, QSPI con XIP y PIO.

Interrupciones. Una interrupción llama a una ISR. Las enmascarables se pueden deshabilitar; las NMI no. Mononivel: todas iguales, una espera a la otra. Multinivel: una de mayor prioridad interrumpe a una menor (anidamiento). NVIC es de ARM Cortex-M; APIC/LAPIC es de x86.

> **Definición de timer (examen)**
> Contador en cuenta regresiva que activa una interrupción al llegar a cero y reinicia automáticamente. **Free-running:** cuenta sin parar. **One-shot:** cuenta una vez, dispara y se detiene.

Memoria. SRAM: flip-flops, rápida, sin refresco. DRAM: capacitores, refresco cada ~60 ms. No volátil: ROM, EEPROM, Flash.

Direccionamiento	Idea clave
Inherente	Sin operandos (NOP).

Direccionamiento	Idea clave
Inmediato	El dato va en la instrucción.
Directo	Dirección corta, página cero.
Extendido	Dirección completa.
Indexado	Registro índice + desplazamiento.
Relativo	Relativo al PC; saltos condicionales.

USB. Árbol con host y hubs, máximo 7 niveles. USB 2.0: Low 1.5 Mbps, Full 12 Mbps, High 480 Mbps. Señal diferencial D+/D−. Por defecto 100 mA a 5 V. Transferencias: Control (configurar), Bulk (grandes datos, sin latencia garantizada), Interrupt (pequeñas, periódicas), Isochronous (tiempo real, sin retransmisión).

Bus	Para recordar
I²C	2 líneas (SDA, SCL), colector abierto con pull-up, maestro-esclavo, direcciones de 7 o 10 bits. START: SDA baja con SCL alto. STOP: SDA sube con SCL alto. ACK/NACK por byte.
UART	Asíncrono, Tx/Rx. N81 = 1 start + 8 datos + 1 stop = **10 bits en el canal**. Flujo: RTS/CTS (hardware), XON/XOFF (software).
SPI	4 hilos (MOSI, MISO, SCK, CS), síncrono, rápido y de corta distancia.
RS-485	Diferencial, largas distancias, multipunto.
One-Wire	Un hilo de datos, parasitic power, ID de 64 bits en ROM.

Arranque. El procesador hace el primer fetch en el **vector de RESET**. Hard RESET (POR) ocurre al aplicar energía; Soft RESET lo causa software, el watchdog, el sobrecalentamiento o ACPI. Orden: estabilizar energía y reloj, fetch del vector de RESET, definir BSP (bit BSP = 1) y AP (bit = 0), el BSP carga BIOS/UEFI e inicializa el APIC, los AP esperan el SIPI, y al final se inicializa la DRAM, se hace POST, se copia el código a RAM y se pasa el control al kernel.